

Atomicity (ACID Property)

1. Overview

Atomicity means a transaction is treated as **one indivisible unit of work**.

None

Either all operations succeed

OR

None of them happen

In simple terms:

None

0% complete OR 100% complete

No partial completion

2. Why Atomicity Matters

Many business operations involve multiple steps.

If some steps succeed and others fail, data becomes corrupted or inconsistent.

Atomicity prevents this.

3. Classic Example: Money Transfer

Transfer ₹50 from Account A to Account B.

Steps:

None

1. Deduct ₹50 from A
2. Add ₹50 to B
3. Commit transaction

Both must happen together.

4. Correct Behavior

Success Case

None

A = 500 → 450
B = 300 → 350

Transaction committed.

Failure Case

Suppose system crashes after deducting from A.

Without atomicity:

None

A = 450

B = 300

₹50 lost.

This is invalid state.

With atomicity:

None

Rollback

A = 500

B = 300

No partial effect remains.

5. Real Meaning in Databases

A transaction containing many SQL statements behaves like one unit.

Example:

SQL

```
BEGIN;
```

```
UPDATE accounts
```

```
SET balance = balance - 50
```

```
WHERE id= 'A';
```

```
UPDATE accounts
```

```
SET balance = balance + 50
```

```
WHERE id= 'B';
```

```
COMMIT;
```

If any step fails:

SQL

```
ROLLBACK;
```

6. Where Atomicity Is Critical

Use cases:

- Banking transfers
- Payment processing
- Order placement
- Inventory deduction
- Ticket booking
- Payroll systems

7. Example: Ecommerce Order

Steps:

None

1. Create order

2. Reduce stock

3. Charge payment

4. Confirm order

Partial completion can cause:

- Payment taken but no order
- Order created but no stock
- Stock reduced without payment

Atomic transactions help avoid this.

8. How Databases Implement Atomicity

Common techniques:

A. Transaction Logs

Changes recorded first.

B. Undo Logs

Revert incomplete work.

C. Rollback Mechanism

Restore previous state.

D. Write-Ahead Logging (WAL)

Used in systems like PostgreSQL and MySQL

9. Atomicity in Distributed Systems

Harder when multiple services involved.

Example:

None

Payment Service

Inventory Service

Order Service

Need patterns like:

- Saga pattern
- Two-phase commit (2PC)
- Compensating transactions

10. Atomicity vs Consistency

Property	Meaning
Atomicity	All or nothing
Consistency	Data remains valid

Atomicity helps consistency, but they are different.

11. Interview Answer Example

Q: Why is atomicity important?

Atomicity ensures multi-step transactions either fully complete or fully roll back, preventing partial updates such as money deducted without being credited elsewhere.

12. Real-World Failure Example

None

User pays ₹1000

Payment success

Server crashes before order creation

Need atomic workflow or compensation.

13. Short Revision Notes

None

Atomicity =

0% or 100%

If one step fails:

rollback all

Examples:

money transfer

payment + order

inventory update

14. One-Line Summary

Atomicity guarantees that a transaction is completed entirely or not at all, preventing partial updates and protecting data integrity.